RAK1906 WisBlock Environmental Sensor Module Quick Start Guide

Prerequisite

What Do You Need?

Before going through each and every step on using the RAK1906 WisBlock module, make sure to prepare the necessary items listed below:

Hardware

- RAK1906 WisBlock Environmental Sensor Module
- Your choice of WisBlock Base
- Your choice of WisBlock Core
- USB Cable
- Li-Ion/LiPo battery (optional)
- Solar charger (optional)

Software

- Download and install Arduino IDE .
- Install the RAKwireless Arduino BSP and add the RAKwireless Core boards to your **Arduino Boards Manager**. Follow the steps in the GitHub repo .

Product Configuration

Hardware Setup

WisBlock can integrate this module, which makes it easy to build up an environmental temperature and humidity data acquisition system.

For more information about the RAK1906, refer to the Datasheet .

The RAK1906 module gives information about:

- Air pressure
- Ambient Air Temperature
- Environment Humidity
- Gas Resistance
- Sensor outputs can be used with the Bosch BSEC library algorithm to calculate the IAQ (Indoor Air Quality) index

RAK1906 module can be connected to the sensor's slot of WisBlock Base to communicate with the WisBlock Core, as shown in **Figure 1**. It will work on **SLOT A to F**. Also, always secure the connection of the WisBlock module by using compatible screws.

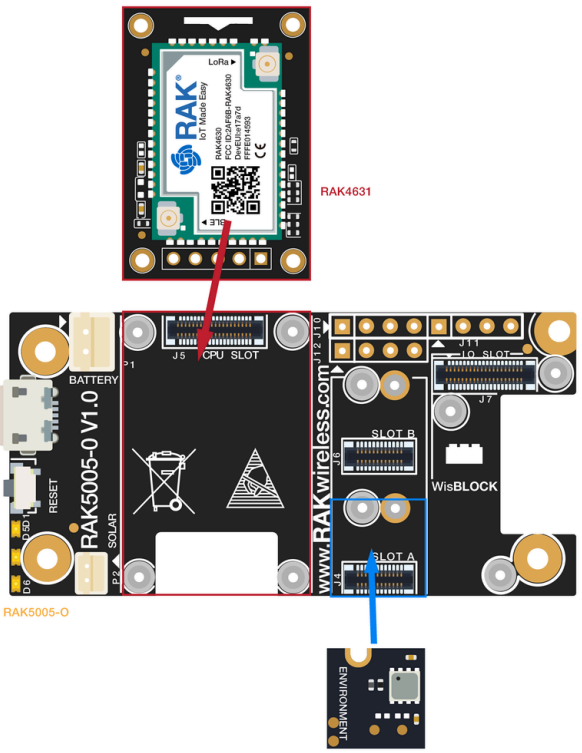


Figure 1: RAK1906 connection to WisBlock Base

Assembling and Disassembling of WisBlock Modules

Assembling

As shown in **Figure 2**, the location for Slot A, B, C, and D are properly marked by silkscreen. Slots C and D are located on bottom of WisBlock Base. Follow carefully the procedure defined in WisBlock Base board assembly/disassembly instructions to attach a WisBlock module. Once attached, carefully fix the module with a M1.2 x 3 mm screw.

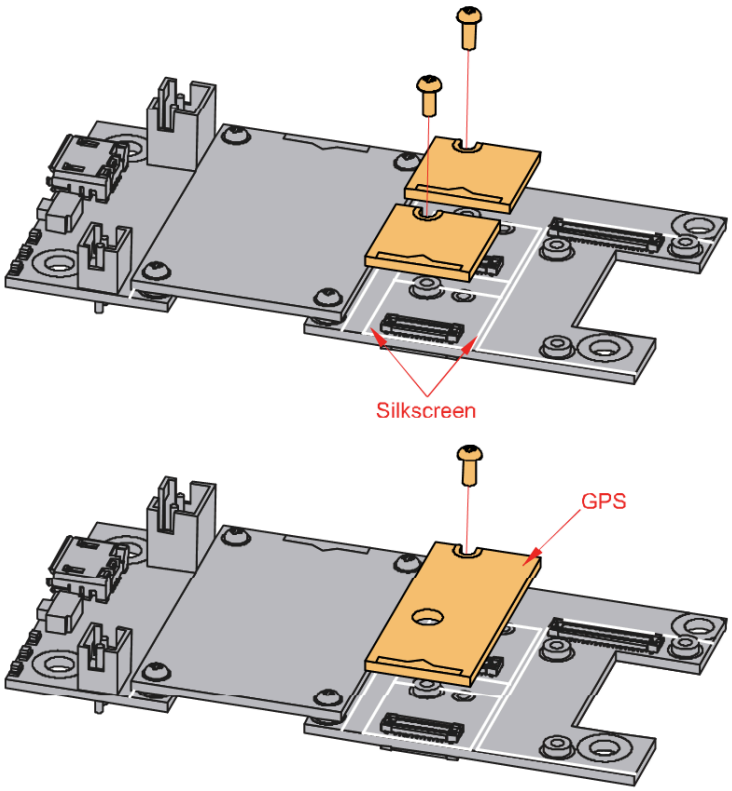


Figure 2: RAK1906 connection to WisBlock Base

Disassembling

The procedure in disassembling any type of WisBlock modules is the same.

1. Remove the screws.

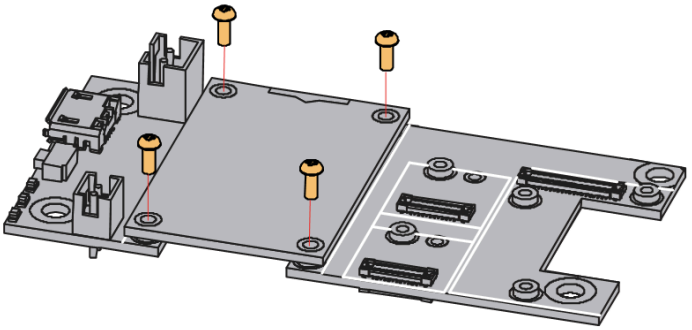


Figure 3: Removing screws from the WisBlock module

2. Once the screws are removed, check the silkscreen of the module to find the correct location where force can be applied.

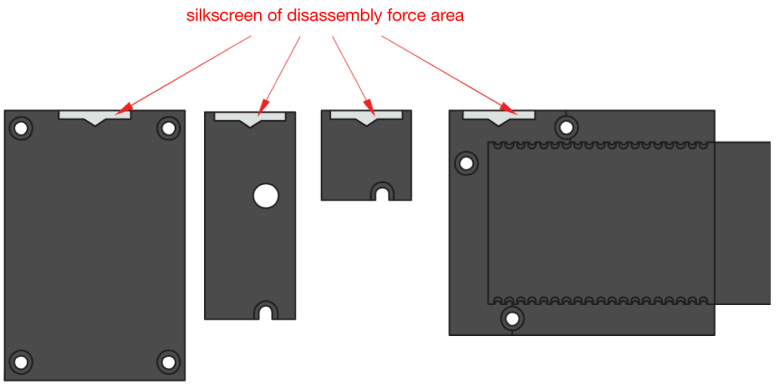


Figure 4: Detaching silkscreen on the WisBlock module

3. Apply force to the module at the position of the connector, as shown in **Figure 5**, to detach the module from the baseboard.

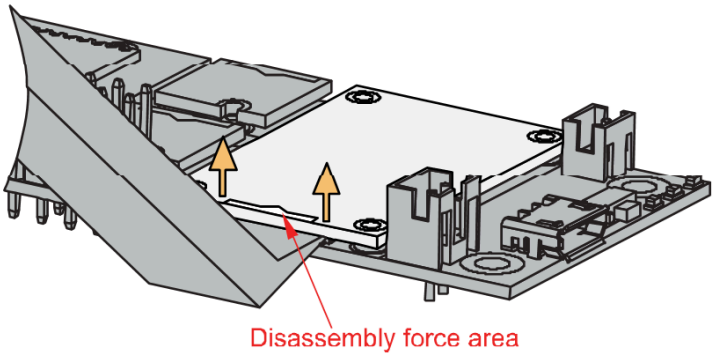


Figure 5: Applying even forces on the proper location of a WisBlock module

NOTE

If you will connect other modules to the remaining WisBlock Base slots, check on the WisBlock Pin Mapper tool for possible conflicts. RAK1906 uses I2C communication lines, and it can cause possible conflict especially on some IO modules.

After all this setup, you can now connect the battery (optional) and USB cable to start programming the RAK1906.

WARNING

- Batteries can cause harm if not handled properly.
- Only 3.7-4.2 V Rechargeable LiPo batteries are supported. It is highly recommended not to use other types of batteries with the system unless you know what you are doing.
- If a non-rechargeable battery is used, it has to be unplugged first before connecting the USB cable to the USB port of the board to configure the device. Not doing so might damage the battery or cause a fire.
- Only 5 V solar panels are supported. Do not use 12 V solar panels. It will destroy the charging unit and eventually other electronic parts.
- Make sure the battery wires match the polarity on the WisBlock Base board. Not all batteries have the same wiring.

Software Configuration and Example

The RAK1906 is a 4-in-1 Environmental Sensor board that contains the Bosch BME680 chip. It is specially designed for wearables where size and low power consumption are critical requirements. The BME680 contains a small Metal-Oxide (MOX) sensor and an I2C interface used to communicate with WisBlock Core.

> **WARNING**
>
> The BME680 requires a burn-in period to collect accurate readings. The burn-in procedure can be achieved by reading all sensors once every 5 seconds for at least 20 minutes the first time that you use it. For each subsequent use, the readings should stabilize 2-3 minutes after power-up.

Initial Test of the RAK1906 WisBlock Module

1. Install the RAKwireless Arduino BSP for WisBlock by using the `package_rakwireless_index.json` board installation package. The WisBlock Core should now be available on the Arduino IDE.

2. You need to select the WisBlock Core you have, as shown in **Figure 6** to **Figure 8**.

RAK4631 WisBlock Core

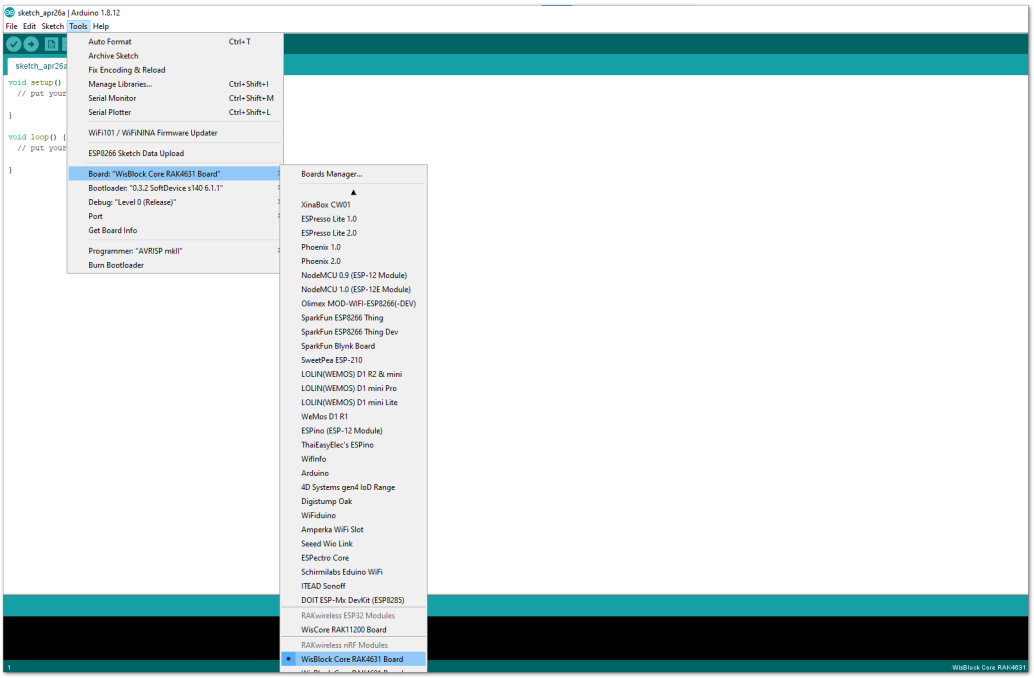


Figure 6: Selecting RAK4631 as WisBlock Core

RAK11200 WisBlock Core

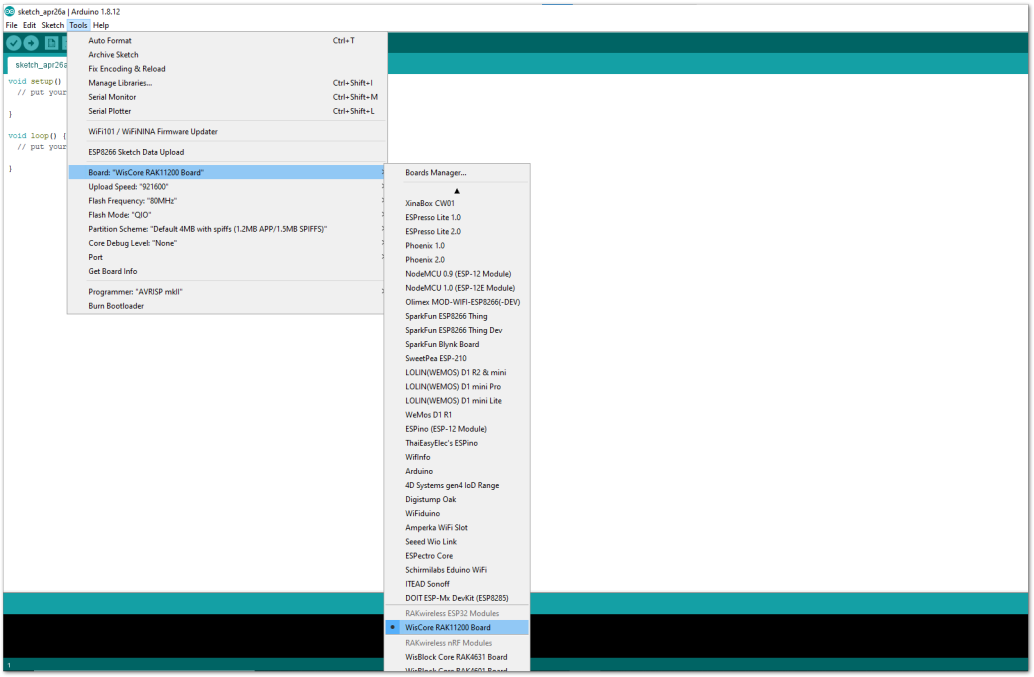


Figure 7: Selecting RAK11200 as WisBlock Core

RAK11310 WisBlock Core

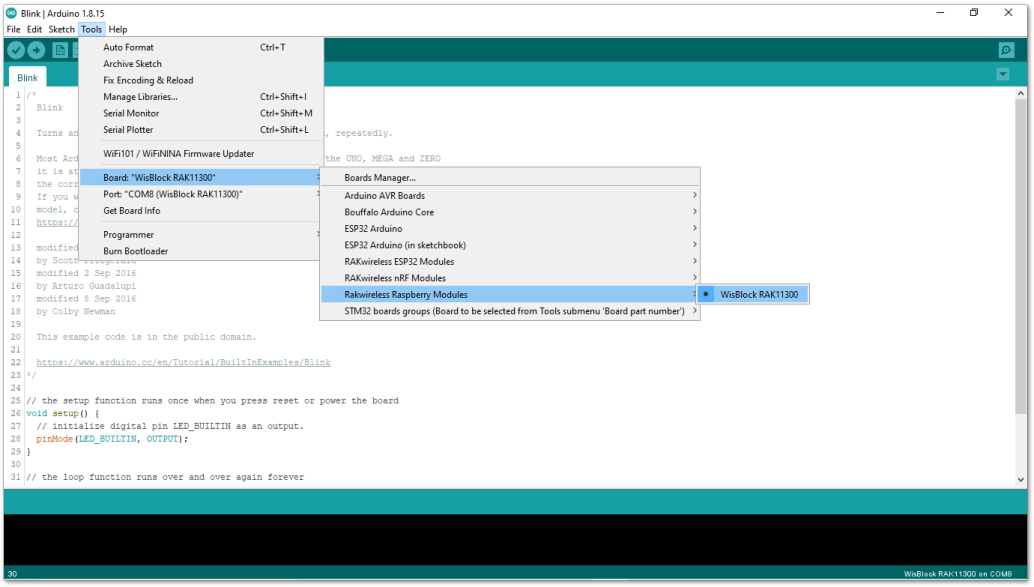


Figure 8: Selecting RAK11310 as WisBlock Core

3. Copy the following sample code into your Arduino IDE:

<details>
<summary>Click to view the code</summary>
</details>

> **NOTE**
>
> If you experience any error in compiling the example sketch, check the updated code for your WisBlock Core Module that can be found on the RAK1906 WisBlock Example Code Repository and this sample code in Github will work on all WisBlock Core.

4. Once the example code is open, install the Adafruit BME680 library by clicking the link highlighted in yellow, as shown in **Figure 9** and **Figure 10**.

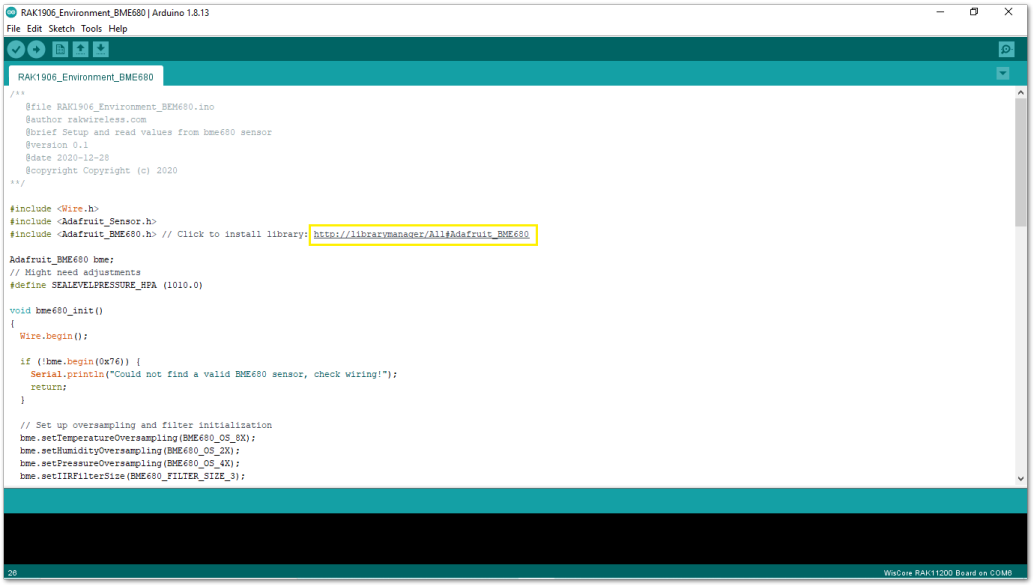


Figure 9: Select Adafruit BME680 library

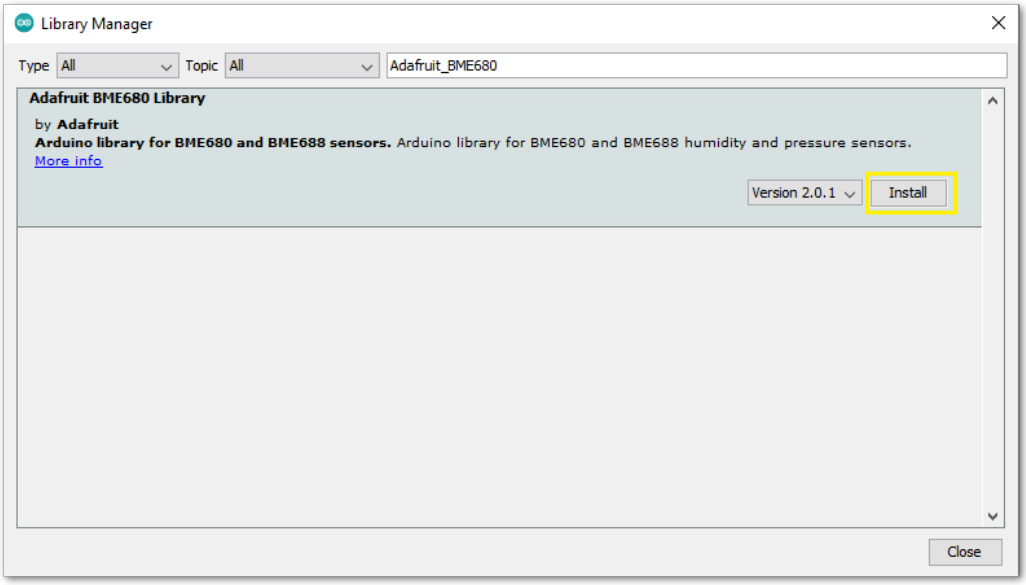


Figure 10: Installing Adafruit BME680 library

During the installation, you will be asked to install the **Adafruit Unified Sensor** library. Install it as well.

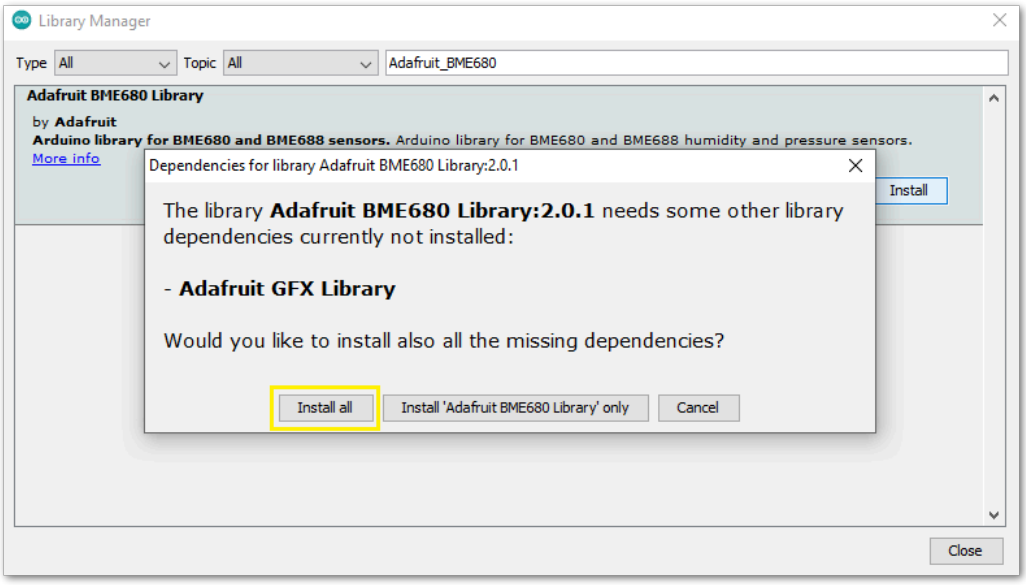


Figure 11: Installing Adafruit Unified Sensor Library

5. After successful installation of the library, you can now select the right serial port and upload the code, as shown in **Figure 12** and **Figure 13**.

> **NOTE**
>
> If you are using the RAK11200 as your WisBlock Core, the RAK11200 requires the **Boot0** pin to be configured properly first before uploading. If not done properly, uploading the source code to RAK11200 will fail. Check the full details on the RAK11200 Quick Start Guide .

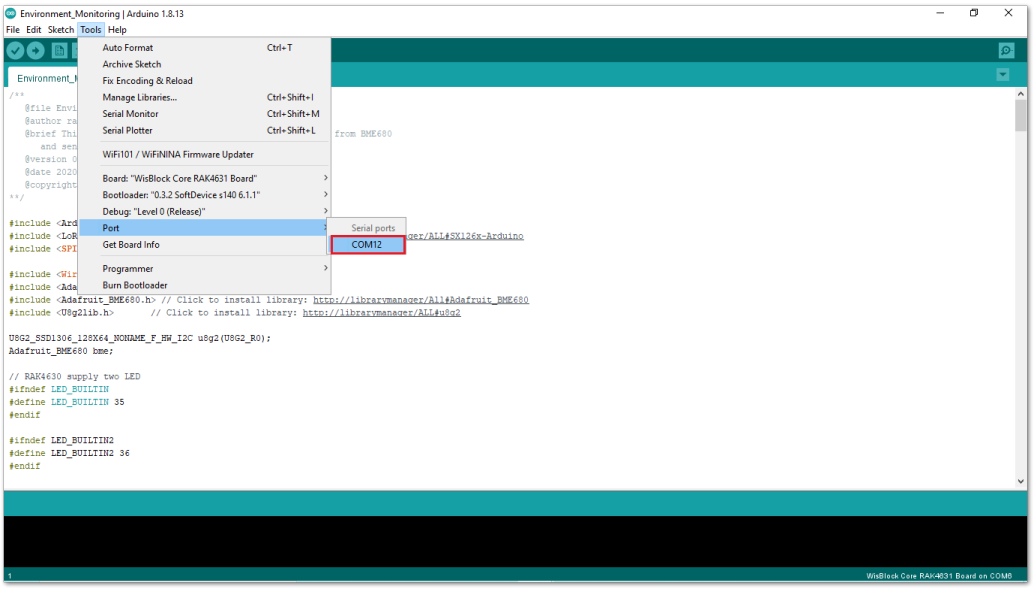


Figure 12: Selecting the correct Serial Port

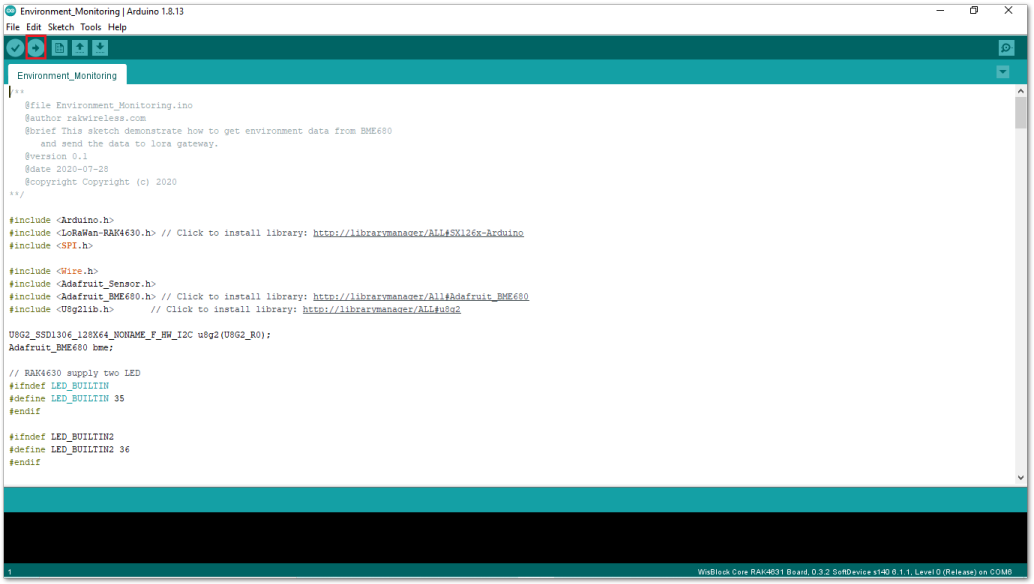


Figure 13: Uploading the RAK1906 example code

6. The RAK1906_Environment_BME680 application gets BME680 sensor data every 5 seconds and prints the results on the serial port.

```
Temperature = 30.66 *C
Pressure = 1013.80 hPa
Humidity = 44.38 %
Gas = 3.89 KOhms

Temperature = 30.69 *C
Pressure = 1013.78 hPa
Humidity = 44.59 %
Gas = 3.88 KOhms

Temperature = 30.72 *C
Pressure = 1013.82 hPa
Humidity = 44.37 %
Gas = 3.87 KOhms
```

LoRaWAN Environment Monitoring with RAK1906

This solution shows how to create an environmental data sensor node. The sensor node measures air temperature, air humidity, barometric pressure, and gas levels. After the node joins to server successfully, it then transmits this data periodically to a LoRaWAN server.

These are the quick links that go directly to the specific WisBlock Core module solution that you can use in your RAK1906 project:

- RAK1906 solution for RAK4631
- RAK1906 solution for RAK11310

RAK4631 Solution Requirements

- 1 pc WisBlock Base board
- 1 pc WisBlock Core RAK4631
- 1 pc WisBlock Sensor RAK1906
- 1 pc WisBlock Display RAK1921
- 1 pc LoRa Gateway
- Arduino IDE
- RAK4630 BSP

- Adafruit BME680 Library
- SX126x-Arduino Library
- U8g2 library

RAK11310 Solution Requirements

- 1 pc WisBlock Base board
- 1 pc WisBlock Core RAK11310
- 1 pc WisBlock Sensor RAK1906
- 1 pc WisBlock Display RAK1921
- 1 pc LoRa Gateway
- Arduino IDE
- Arduino BSP for RAK11300 module
- Adafruit BME680 Library
- SX126x-Arduino Library
- U8g2 library

Configuring LoRaWAN

There are configurations that you need to setup to ensure that the device can join a LoRaWAN Network server.

The steps below will explain the default settings and how to configure it.

1. Set up the LoRa Region.

Default region is EU868.

```
LoRaMacRegion_t g_CurrentRegion = LORAMAC_REGION_EU868;
```

You can change this to a region that is applicable to you like `LORAMAC_REGION_US915`, `LORAMAC_REGION_AU915`, etc.

2. Set up the LoRaWAN activation method.

Default is **OTAA**.

```
bool doOTAA = true;
```

To configure the device to ABP, you need to make this boolean variable `false`.

3. Set up the message type if confirmed or not.

Default is **confirmed message**.

```
lmh_confirm g_CurrentConfirm = LMH_CONFIRMED_MSG;
```

You can change to unconfirmed message by changing the value to `LMH_UNCONFIRMED_MSG`.

4. Set up device class.

Default is **Class A**.

```
DeviceClass_t g_CurrentClass = CLASS_A;
```

You can change this to **CLASS_B** (still under development) or **CLASS_C**.

5. Set up the keys.

- Configuration keys for OTAA Activation:

```
uint8_t nodeDeviceEUI[8] = {0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x33, 0x33};
uint8_t nodeAppEUI[8] = {0xB8, 0x27, 0xEB, 0xFF, 0xFE, 0x39, 0x00, 0x00};
uint8_t nodeAppKey[16] = {0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88};
```

- Configuration keys for ABP Activation:

```
uint32_t nodeDevAddr = 0x260116F8;
uint8_t nodeNwsKey[16] = {0x7E, 0xAC, 0xE2, 0x55, 0xB8, 0xA5, 0xE2, 0x69, 0x91, 0x51, 0x96, 0x06, 0x47, 0x56, 0x9D, 0x23};
uint8_t nodeAppsKey[16] = {0xFB, 0xAC, 0xB6, 0x47, 0xF3, 0x58, 0x45, 0xC7, 0x50, 0x7D, 0xBF, 0x16, 0x8B, 0xA8, 0xC1, 0x7C};
```

6. Set up uplink period.

The default is **20000 mS**.

```
#define LORAWAN_APP_INTERVAL 20000
```

Data Format

Byte1	Byte2	Byte3	Byte4	Byte5
1	Temperature1	Temperature2	Humidity1	Humidity2

Byte6	Byte7	Byte8	Byte9	Byte10	Byte11	Byte12	Byte13
Barometric1	Barometric2	Barometric3	Barometric4	Gas1	Gas2	Gas3	Gas4

- Byte1 is a marker for the data type, here always 1.
- Every byte value is MSB first.
- Temperature is sent as two bytes, enlarged 100 times for accuracy. For example, 2510 means 25.01 C, sent as 0x09 0xCE.
- Humidity is sent as two bytes, enlarged 100 times for accuracy. For example, 4173 means 41.73%, sent as 0x10 0x4D.
- Barometric pressure is sent as four bytes, enlarged 100 times for accuracy. For example, 100945 means 1009.45 hPa, sent as 0x00 0x01 0x8A 0x51.
- Gas resistance is sent as four bytes. For example, 63560 Ohm, sent as 0x00 0x00 0xF8 0x48.

Attention

If this example is implemented for the Region US915, DR0 cannot be used because the package size is too large to fit into the allowed payload.

When you successfully uploaded the example sketch, open the Serial Monitor of the Arduino IDE to see the sensor's reading logs. If you see the logs, as shown below, then your RAK1906 is properly communicating to the WisBlock core.

```
10:30:07.777 -> ====================================
10:30:07.777 -> Welcome to RAK4630 LoRaWan!!!
10:30:07.777 -> Type: OTAA
10:30:07.777 -> Region: EU868
10:30:07.777 -> ====================================
10:30:07.777 -> Chip ID=0x61
10:30:12.974 -> OTAA Mode, Network Joined!
10:30:33.999 -> Sending frame now...
10:30:33.999 -> result: Tem:28.70C Hum:67.00% Pres:948.50KPa Gas:27646Ohms
10:30:33.999 -> lmh_send ok count 1
10:30:53.979 -> Sending frame now...
```

```
10:30:53.979 -> result: Tem:28.70C Hum:67.00% Pres:948.50KPa Gas:242488Ohms
10:30:54.012 -> lmh_send ok count 2
10:31:14.006 -> Sending frame now...
10:31:14.006 -> result: Tem:28.70C Hum:67.00% Pres:948.50KPa Gas:457174Ohms
10:31:14.006 -> lmh_send ok count 3
10:31:33.997 -> Sending frame now...
10:31:33.997 -> result: Tem:28.69C Hum:67.00% Pres:948.50KPa Gas:74102Ohms
10:31:34.032 -> lmh_send ok count 4
```

Data arrives at LoRaWAN server.



Figure 14: LoRaWAN Environment Monitoring example